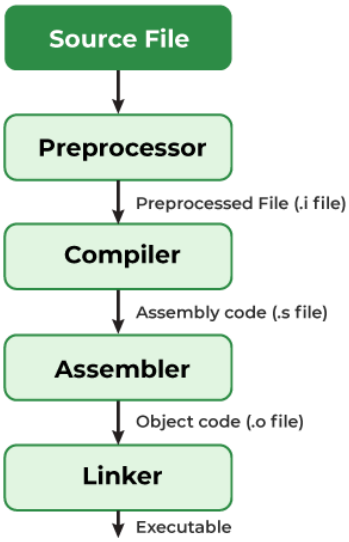
C Preprocessors

Last Updated : 03 Jun, 2025

Preprocessors are programs that process the source code before the actual compilation begins. They are not part of the compilation process but operate separately, allowing programmers to modify the code before compilation. It is the first step that the C source code goes through when being converted into an executable file.



Preprocessor Directives

The **preprocessor directives** are the instructions to the preprocessor for doing some tasks such as text substitutions, macro expansions, including header files, and many more before compiling the code. All of these preprocessor directives begin with a '**#**' **hash** symbol.

One of the most commonly used preprocessors is **#include** which is used to include the header files for different libraries in C programs. C provides more directives for different functionalities.

List of Preprocessor Directives

The following table lists all the preprocessor directives in C:

Preprocessor Directives	Description
#define	Used to define a macro
#undef	Used to undefine a macro
#include	Used to include a file in the source code program
#ifdef	Used to include a section of code if a certain macro is defined by #define
#ifndef	Used to include a section of code if a certain macro is not defined by #define
#if	Check for the specified condition
#else	Alternate code that executes when #if fails
#elif	Combines else and if for another condition check
#endif	Used to mark the end of #if, #ifdef, and #ifndef

These preprocessors can be classified based on the type of function they perform.

Types of C Preprocessors

All the above preprocessors can be classified into 4 types:

Table of Content

- Macros
- File Inclusion
- Conditional Compilation

- Other Directives

Let us now learn about each of these directives in detail.

Macros

Macros are used to define constants or create functions that are substituted by the preprocessor before the code is compiled. The two preprocessors **#define** and **#undef** are used to create and remove macros in C.

> ***#define*** *token value*
> ***#undef*** *token*

where after preprocessing, the ***token*** will be expanded to its ***value*** in the program.

Example:

```c
#include <stdio.h>

// Macro Definition
#define LIMIT 5

int main(){
    for (int i = 0; i < LIMIT; i++) {
        printf("%d \n", i);
    }
    return 0;
}
```

Output

```
0
1
2
3
4
```

In the above program, before the compilation begins, the word LIMIT is replaced with 5. The word **'LIMIT'** in the macro definition **is called a macro template** and **'5' is macro expansion.**

> ***Note*** *There is no semi-colon (;) at the end of the macro definition. Macro definitions do not need a semi-colon to end.*

There are also some Predefined Macros in C which are useful in providing various functionalities to our program.

A macro defined previously can be undefined using #undef preprocessor. For example, in the above code,

```c
#include <stdio.h>

// Macro Definition
#define LIMIT 5

// Undefine macro
#undef LIMIT

int main(){
    for (int i = 0; i < LIMIT; i++) {
        printf("%d \n", i);
    }
    return 0;
}
```

Output:

```
./Solution.c: In function 'main':
./Solution.c:13:28: error: 'MAX' undeclared (first use in this
function)
     printf("MAX is: %d\n", MAX);
                            ^
./Solution.c:13:28: note: each undeclared identifier is reported only
once for each function it appears in
```

Macros With Arguments

We can also pass arguments to macros. These macros work similarly to functions. **For example,**

*#**define** foo(a, b) a + b*
*#define func(r) r * r*

Let us understand this with a program:

```c
#include <stdio.h>
```

```c
// macro with parameter
#define AREA(l, b) (l * b)

int main(){
    int a = 10, b = 5;

    // Finding area using above macro
    printf("%d", AREA(a, b));
    return 0;
}
```

Output

```
Area of rectangle is: 50
```

Explanation: In the above program, the macro **AREA(l, b)** is defined to calculate the area of a rectangle by multiplying its **length (l)** and **breadth (b)**. When **AREA(a, b)** is called, it expands to **(a * b)**, and the result is computed and printed.

File Inclusion

File inclusion allows you to include external files (header files, libraries, etc.) into the current program. This is typically done using the **#include** directive, which can include both system and user-defined files.

Syntax

There are two ways to include header files.

#include *<file_name>*
#include *"filename"*

The **'<'** and **'>' brackets** tell the compiler to look for the file in the **standard directory** while **double quotes (" ")** tell the compiler to search for the header file in the source file's directory.

Example:

```c
// Includes the standard I/O library
#include <stdio.h>

int main() {
    printf("Hello World");
```

```
    return 0;
}
```

Output

```
Hello World
```

Conditional Compilation

Conditional compilation allows you to include or exclude parts of the code depending on certain conditions. This is useful for creating platform-specific code or for debugging. There are the following conditional preprocessor directives: #if, #ifdef, #ifndef, else, #elif and #endif

Syntax

The general syntax of conditional preprocessors is:

```
#if
  // some code
#elif
  // some more code
#else
  // Some more code
#endif
```

#endif directive is used to close off the #if, #ifdef, and #ifndef opening directives.

Example

```c
#include <stdio.h>

// Defining a macro for PI
#define PI 3.14159

int main(){

// Check if PI is defined using #ifdef
#ifdef PI
    printf("PI is defined\n");

// If PI is not defined, check if SQUARE is defined
#elif defined(SQUARE)
    printf("Square is defined\n");
```

```
// If neither PI nor SQUARE is defined, trigger an error
#else
    #error "Neither PI nor SQUARE is defined"
#endif

// Check if SQUARE is not defined using #ifndef
#ifndef SQUARE
    printf("Square is not defined");

// If SQUARE is defined, print that it is defined
#else
    printf("Square is defined");
#endif

    return 0;
}
```

Output

```
PI is defined
Square is not defined
```

Explanation: This code uses conditional preprocessor directives (**#ifdef, #elif, and #ifndef**) to check whether certain macros (**PI** and **SQUARE**) are defined. Since PI is defined, the program prints "**PI is defined**", then checks if SQUARE is not defined and prints "**Square is not defined**".

Other Directives

Apart from the primary preprocessor directives, C also provides other directives to manage compiler behaviour and debugging.

#pragma:

Provides specific instructions to the compiler to control its behaviour. It is used to disable warnings, set alignment, etc.

Syntax

#pragma directive

Some of the #pragma directives are discussed below:

1. **#pragma startup:** These directives help us to specify the functions that are needed to run before program startup (before the control passes to

main()).

2. **#pragma exit**: These directives help us to specify the functions that are needed to run just before the program exit (just before the control returns from main()).

Example

```
#include <stdio.h>
void func1();
void func2();

// specifying funct1 to execute at start
#pragma startup func1

// specifying funct2 to execute before end
#pragma exit func2

void func1() { printf("Inside func1()\n"); }
void func2() { printf("Inside func2()\n"); }
int main(){
    void func1();
    void func2();
    printf("Inside main()\n");

    return 0;
}
```

Output

```
Inside main()
```

The above code will produce the output as given above when run on GCC compilers while the expected output was:

Expected Output

```
Inside func1()
Inside main()
Inside func2()
```

This happens because GCC does not support #pragma startup or exit. However, you can use the below code for the expected output on GCC compilers.

```
#include <stdio.h>
```

```c
#include <stdio.h>

void func1();
void func2();

void __attribute__((constructor)) func1();
void __attribute__((destructor)) func2();

void func1()
{
    printf("Inside func1()\n");
}

void func2()
{
    printf("Inside func2()\n");
}

int main()
{
    printf("Inside main()\n");

    return 0;
}
```

Output

```
Inside func1()
Inside main()
Inside func2()
```

In the above program, we have used some specific syntaxes so that one of the functions executes before the main function and the other executes after the main function.

#pragma warn Directive

This directive is used to hide the warning message which is displayed during compilation. We can hide the warnings as shown below:

- **#pragma warn -rvl**: This directive hides those warnings which are raised when a function that is supposed to return a value does not return a value.

- **#pragma warn -par**: This directive hides those warnings which are raised when a function does not use the parameters passed to it.
- **#pragma warn -rch**: This directive hides those warnings which are raised when a code is unreachable. For example, any code written after the *return* statement in a function is unreachable.

Comment More info Advertise with us

Next Article

C Preprocessor Directives

Similar Reads

C Preprocessors

Preprocessors are programs that process the source code before the actual compilation begins. They are not part of the compilation process but operate separately, allowing programmers to modify the code...

8 min read

C Preprocessor Directives

In C programming, the preprocessor is a program that process the source code before the actual compilation begins. It uses preprocessor directives are commands that instruct the preprocessor to perfor...

6 min read

How a Preprocessor works in C?

Compiling a C program - Behind the Scene A Preprocessor is a system software (a computer program that is designed to run on computer's hardware and application programs). It performs preprocessing of the...

3 min read

Header Files in C

In C programming, a header file is a file that ends with the .h extension and contains features like functions, data types, macros, etc that can be used by any other C program by including that particular...

5 min read

Whatâ€™s difference between header files "stdio.h" and "stdlib.h" ?

In C programming, standard header files provide various inbuilt functionalities and two of the most commonly used standard header files are stdio.h and stdlib.h. The <stdio.h> provides Standard Input...

4 min read

How to write your own header file in C?

As we all know that files with .h extension are called header files in C. These header files generally contain function declarations which we can be used in our main C program, like for e.g. there is need to...

4 min read

Macros and its types in C

In C programming, a macro is a symbolic name or constant that represents a value, expression, or code snippet. They are defined using the #define directive, and when encountered, the preprocessor substitute...

4 min read

Interesting Facts about Macros and Preprocessors in C

In a C program, all lines that start with # are processed by preprocessor which is a special program invoked by the compiler. by this we mean to say that the â€˜#â€™ symbol is used to process the...

5 min read

and ## Operators in C

In C, # and ## operators are preprocessor operators using in macros for token manipulation. They are known as stringizing and token pasting operators and are used in macro definition with #define...

3 min read

How to print a variable name in C?

Printing a variable name means printing the identifier that is assigned to the variable. To print it, it should be in the form of string. This can be done by using stringification.Stringification in C is the method to...

1 min read

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

GET IT ON Google Play
Download on the App Store

Advertise with us

Contact Us
Advertise with us
GFG Corporate Solution
Placement Training Program

GoLang
SQL
R Language
Android Tutorial
Tutorials Archive

DSA

Data Structures
Algorithms
DSA for Beginners
Basic DSA Problems
DSA Roadmap
Top 100 DSA Interview Problems
DSA Roadmap by Sandeep Jain
All Cheat Sheets

Data Science & ML

Data Science With Python
Data Science For Beginner
Machine Learning
ML Maths
Data Visualisation
Pandas
NumPy
NLP
Deep Learning

Web Technologies

HTML
CSS
JavaScript
TypeScript
ReactJS
NextJS
Bootstrap
Web Design

Python Tutorial

Python Programming Examples
Python Projects
Python Tkinter
Python Web Scraping
OpenCV Tutorial
Python Interview Question
Django

Computer Science

Operating Systems
Computer Network
Database Management System
Software Engineering
Digital Logic Design
Engineering Maths
Software Development
Software Testing

DevOps

Git
Linux
AWS
Docker
Kubernetes
Azure
GCP
DevOps Roadmap

System Design

High Level Design
Low Level Design
UML Diagrams
Interview Guide
Design Patterns
OOAD
System Design Bootcamp
Interview Questions

Inteview Preparation

Competitive Programming
Top DS or Algo for CP
Company-Wise Recruitment Process
Company-Wise Preparation
Aptitude Preparation
Puzzles

School Subjects

Mathematics
Physics
Chemistry
Biology

GeeksforGeeks Videos

DSA
Python
Java
C++

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved